

C PROGRAMMING INTRODUCTION



TUẦN 12: MẢNG VÀ CON TRỎ

**Trường Công nghệ Thông tin và Truyền thông
Trường Đại học Bách Khoa Hà Nội**

Mảng và con trỏ

- Mảng **S** có địa chỉ là địa chỉ của phần tử đầu tiên của mảng **S[0]**
- **S** là một con trỏ tới **S[0]**
int S[10];
int *iptr;
iptr = S; /* From now iptr is equivalent to s */
- Cả **iptr** và **S** đều trỏ tới **S[0]**

Bài thực hành 1

```
#include <stdio.h>
```

```
int main() {
```

```
    int x[4];
```

```
    int i;
```

```
    for(i = 0; i < 4; ++i) {
```

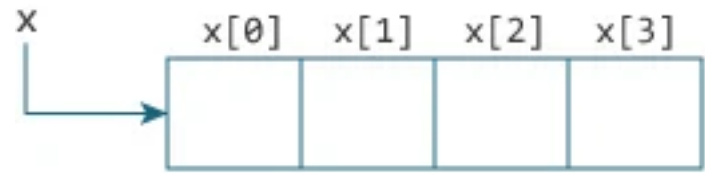
```
        printf("&x[%d] = %p\n", i, &x[i]);
```

```
    }
```

```
    printf("Address of array x: %p", x);
```

```
    return 0;
```

```
}
```



```
&x[0] = 000000000065FE00
```

```
&x[1] = 000000000065FE04
```

```
&x[2] = 000000000065FE08
```

```
&x[3] = 000000000065FE0C
```

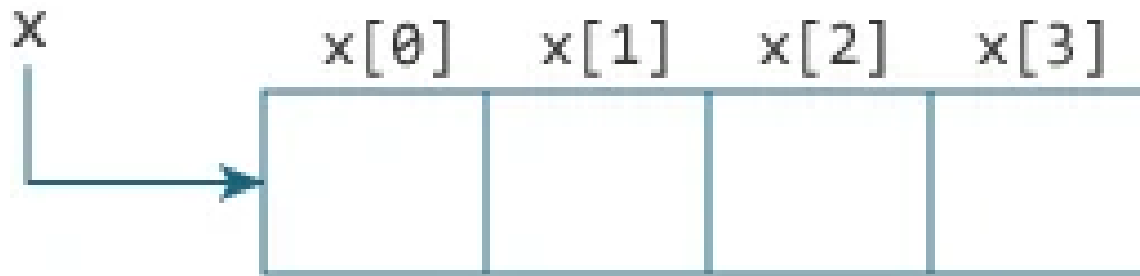
```
Address of array x:
```

```
000000000065FE00
```

Yêu cầu: Hãy chạy chương trình, ghi lại kết quả và giải thích kết quả thu được.

Thao tác với con trỏ

- Có thể tăng hoặc giảm con trỏ
- Nếu $\mathbf{p}$ là một con trỏ có kiểu xác định, $\mathbf{p}+1$ trỏ tới địa chỉ của biến tiếp theo có cùng kiểu
- $\mathbf{p}++$, $\mathbf{p}+\mathbf{i}$, và $\mathbf{p} += \mathbf{i}$ đều có thể được sử dụng
- Nếu $\mathbf{p}$ và $\mathbf{q}$ trỏ đến các phần tử trong mảng, $\mathbf{q}-\mathbf{p}$ cho ra số phần tử nằm giữa $\mathbf{p}$ và $\mathbf{q}$.



Bài thực hành 2

Yêu cầu: Hãy chạy chương trình, ghi lại kết quả và giải thích kết quả thu được.

```
int main(void)
{
    int a[3] = {17,289,4913};
    int *p, *q;

    p = a;    /* p points to the beginning of a, that is &a[0] */
    q = p+2;  /* q points to a[2]. Equivalent to q = &a[2] */

    printf("a is %p\n", a);
    printf("p is %p, q is %p\n", p, q);
    printf("p points to %d and q points to %d\n", *p, *q);
    printf("The pointer distance between p and q is %ld\n", q-p);
    printf("The integer distance between p and q is %ld\n",
           (long int)q - (long int)p);
    return 0;
}
```

Bài thực hành 3

Yêu cầu: Hãy chạy chương trình, ghi lại kết quả và giải thích kết quả thu được.

```
#include <stdio.h>
int main() {
    int i, x[3], sum = 0;
    printf("Enter 3 numbers: ");
    for(i = 0; i < 3; ++i) {
        // Equivalent to scanf("%d", &x[i]);
        scanf("%d", x+i);
        // Equivalent to sum += x[i]
        sum += *(x+i);
    }
    printf("Sum = %d", sum);
    return 0;
}
```

Bài thực hành 4

```
#include <stdio.h>
```

```
int main() {
```

```
    int x[5] = {1, 2, 3, 4, 5};
```

```
    int* ptr;
```

```
    // ptr is assigned the address of the third element
```

```
    ptr = &x[2];
```

```
    printf("*ptr = %d \n", *ptr);
```

```
    printf("*(ptr+1) = %d \n", *(ptr+1));
```

```
    printf("*(ptr+1) = %d \n", ptr[1]);
```

```
    printf("*(ptr-1) = %d", *(ptr-1));
```

```
    printf("*(ptr-1) = %d", ptr[-1]);
```

```
    return 0;
```

```
}
```

Yêu cầu:

- Hãy chạy chương trình, ghi lại kết quả và giải thích kết quả thu được.
- Hãy bổ sung mã hiển thị giá trị của ptr, ptr+1, ptr-1

Bài thực hành 5. Truyền mảng cho hàm

- HUSTack

- Có thể truyền mảng cho hàm bằng con trỏ
- Bài thực hành 5: Hàm tính tổng các phần tử của một mảng
- Viết hàm main sử dụng hàm addNumbers: trong hàm main, khai báo mảng X là mảng số nguyên có 5 phần tử, nhập dữ liệu cho mảng này từ bàn phím, gọi hàm addNumbers với mảng X để trả về biến S, in kết quả của biến S ra màn hình.

```
#include <stdio.h>

int addNumbers(int *fiveNumbers) {
    int i, sum=0;
    for(i=0; i<5; i++, fiveNumbers++) {
        sum+= *fiveNumbers;
    }
    return sum;
}
```

Bài thực hành 6

- Hãy mô tả chức năng của hàm countEven.
- Viết hàm main sử dụng hàm countEven :
 - Trong hàm main, khai báo mảng X là mảng số nguyên, mảng này sẽ có N phần tử với N là một số nguyên dương nhập từ bàn phím.
 - Nhập dữ liệu cho mảng này từ bàn phím.
 - Gọi hàm countEven với mảng X và biến N để trả về biến S, in kết quả của biến S ra màn hình.

```
int countEven(int* arr, int size) {  
    int i;  
    int count = 0;  
    for (i=0; i<size; i++)  
        if (*(arr+i) % 2 == 0) count++;  
    return count;  
}
```

Bài thực hành 7: Hàm trả về con trỏ

- HUSTack

- Viết hàm `maximum` trả về con trỏ trỏ đến số lớn nhất trong một mảng `double`. Nếu mảng rỗng, trả về `NULL`.

`double* maximum(double* a, int size);`

- Viết hàm `main` sử dụng hàm `maximum` :
 - Trong hàm `main`, khai báo mảng `X` là mảng số nguyên có `N` phần tử với `N` là một số nguyên dương nhập từ bàn phím.
 - Nhập dữ liệu cho mảng này từ bàn phím.
 - Gọi hàm `maximum` với mảng `X` và biến `N` để trả về biến `pS`, in kết quả của `pS` và `*pS` ra màn hình. Nhận xét kết quả thu được.

Bài thực hành 8 - HUSTack

- Viết hàm ***getSales*** nhận vào tham số là một mảng và số phần tử của mảng là số quý trong năm và yêu cầu người dùng nhập vào doanh số mỗi quý
- Viết hàm ***totalSales*** trả về doanh số trong cả năm
- Sử dụng hai hàm này trong chương trình
- Sử dụng con trỏ để truyền tham số vào cho hàm

Bài thực hành 9: Array of pointers

```
#include <stdio.h>
int main() {
    int* ptrarray[4];
    int w = 100, x = 200, y = 300, z = 400;
    ptrarray[0] = &w;
    ptrarray[1] = &x;
    ptrarray[2] = &y;
    ptrarray[3] = &z;
    for (int i = 0; i < 4; i++) {
        printf("The value %d has the address %p\n",
               *ptrarray[i], ptrarray[i]);
    }
    return 0;
}
```

Yêu cầu: Hãy chạy chương trình, ghi lại kết quả và giải thích kết quả thu được.

HUSTack

Bài thực hành 10: Array of pointers

```
#include <stdio.h>
const int MAX = 3;
int main () {
    int var[] = {10, 100, 200};
    int i, *ptr[MAX];
    for ( i = 0; i < MAX; i++) {
        ptr[i] = &var[i]; /* assign the address of integer. */
    }
    for ( i = 0; i < MAX; i++) {
        printf("Value of var[%d] = %d\n", i, *ptr[i] );
    }
    return 0;
}
```

Yêu cầu a: Hãy chạy chương trình, ghi lại kết quả và giải thích kết quả thu được.

HUSTack

$$s_x = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2}$$

Yêu cầu b: Hãy bổ sung mã để hiển thị nội dung các phần tử của mảng ptr và biến var. So sánh và nhận xét mối quan hệ giữa các phần tử của mảng ptr và biến var.

Bài thực hành 11

```
#include <stdio.h>
const int MAX = 4;
int main () {
    char *names[] = {
        "Zara Ali",
        "Hina Ali",
        "Nuha Ali",
        "Sara Ali"
    };
    int i = 0;
    for ( i = 0; i < MAX; i++) {
        printf("Value of names[%d] = %s\n", i, names[i] );
    }
    return 0;
}
```

Yêu cầu:

- Hãy chạy chương trình, ghi lại kết quả và giải thích kết quả thu được.
- Hãy bổ sung mã hiển thị giá trị của các phần tử trong mảng names.
- Hãy bổ sung code hiển thị chữ cái đầu và địa chỉ của chữ cái đầu của từng xâu trên.

Bài thực hành 12

- Giải thích chi tiết hàm `reversearray` đảo ngược mảng sử dụng:
 - Chỉ số
 - Con trỏ
- Viết hàm main sử dụng hàm `reversearray`:
 - Nhập một mảng N số nguyên từ bàn phím với N nhập từ bàn phím.
 - Gọi hàm trên để đảo ngược mảng.
 - In kết quả mảng đã đảo ngược ra màn hình

Lời giải

```
void reversearray(int arr[], int size) {  
    int i, j, tmp;  
    i = 0; j = size - 1;  
    while(i < j) {  
        tmp = a[i];  
        a[i] = a[j];  
        a[j] = tmp;  
        i++; j--;  
    }  
}
```

Lời giải

```
void reversearray(int *arr, int size) {  
    int i, j, tmp;  
    i = 0; j = size - 1;  
    while(i < j) {  
        tmp = *(a+i);  
        *(a+i) = *(a+j);  
        *(a+j) = tmp;  
        i++; j--;  
    }  
}
```

Bài thực hành 13

- Xác định kết quả của chương trình sau và giải thích kết quả thu được:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int arr[] = {1, 2, 3, 4, 5};
```

```
    int *p = arr;
```

```
    printf("%d ", ++*p);
```

```
    p += 2;
```

```
    printf("%d ", *p);
```

```
    return 0;
```

```
}
```

```
#include <stdio.h>
```

```
void f(char**);
```

```
int main()
```

```
{
```

```
    char *argv[] = { "ab", "cd", "ef", "gh", "ij", "kl" };
```

```
    f(argv);
```

```
    return 0;
```

```
}
```

```
void f(char **p)
```

```
{
```

```
    char *t;
```

```
    t = (p += sizeof(int))[-1];
```

```
    printf("%s\n", t);
```

```
}
```

Bài thực hành 14

Yêu cầu:

- Hãy chạy chương trình, ghi lại kết quả và giải thích kết quả thu được.
- Hãy bổ sung mã hiển thị giá trị của các phần tử trong mảng argv.
- Hãy bổ sung code hiển thị nội dung của biến t.

```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
    char str1[ ] = "trytoprogram";
```

```
    char str2[ 50 ];
```

```
    char *ptr1 = "hello world";
```

```
    char *ptr2;
```

```
    //look carefully
```

```
    str2 = str1; //error
```

```
    ptr2 = ptr1; //valid
```

```
    str1 = "program"; //error because it is defined and initialized  
already
```

```
    ptr1 = "program"; //valid and works
```

```
    printf("Xau ptr 1 : %s \t Xau ptr2: %s \n", ptr1, ptr2);
```

```
    return 0;
```

```
}
```

Bài thực hành 15

Yêu cầu: Hãy chạy chương trình, ghi lại kết quả và giải thích kết quả thu được.

Bài thực hành 16

- Xác định kết quả của chương trình sau và giải thích kết quả thu được:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char *ptr = "ABCDE";
```

```
    printf("%c", *&*&*ptr);
```

```
    return 0;
```

```
}
```

Bài thực hành 17: Con trỏ void

- Tìm lỗi sai của chương trình sau và sửa lỗi

```
#include <stdio.h>
```

```
int main() {
```

```
    int i, N = 5;
```

```
    float X = 3.5;
```

```
    char A[4] = {'a','b','c','d'};
```

```
    void *pi, *pf; void *pc = A;
```

```
    pi = &N; pf = &X;
```

```
    printf("%p\n%p\n", pi, pf);
```

```
    for (i=0; i<4; i++) {
```

```
        printf("%d ",*(pc++));
```

```
    }
```

```
    return 0;
```

```
}
```

Bài thực hành 18

Duyệt mảng sử dụng kiểu con trỏ

```
char A[4] = {'a', 'b', 'c', 'd'};  
char *pTmp;  
void *pC = A;  
pTmp = (char*)pC;  
for (i=0; i<4; i++){  
    printf("%c ", *(pTmp++));  
}
```